> **How to use this:** each part of the system gets a short walkthrough plus the questions you'll be asked, with short verified answers. Compiled from the live code on 4 Oct 2026 and the project's own **Docs/** folder — every answer here traces back to a file, an endpoint or a table.

1 The whole system at a glance

ARCHITECTURE

- **Three repos, three jobs: Team22-API** is the Express API + PostgreSQL + the docker-compose that runs everything; **Team22-Main** is the web front-end (plus the Docs/ folder); **Team22-Mobile** is the same front-end wrapped into an Android APK.
- **Three containers:** studio_booking_db (Postgres 15, database *semsequipmentbooking*), studio_booking_api (Node/Express on port 5000), studio_booking_frontend (nginx serving the built React app on port 80). One command starts the stack: *docker compose up -d* in Team22-API; helper scripts start-studio.ps1 / check-studio.ps1 watch it.
- **One origin:** the browser only ever talks to nginx; nginx proxies */api/* and */uploads/* to the API — which is why the web app needs no CORS handling and image URLs stay relative.
- **Five roles:** Student, Lecturer, Studio Staff, Admin, Security — each with its own sidebar, landing page and allowed routes. Enforcement happens twice: in the UI (ProtectedRoute) and in the API (verifyToken + role checks).

```
browser → apiFetch() → /api/… → nginx → Express (src/app.js)
→ verifyToken (JWT) → requireRole([…]) → controller → PostgreSQL
```

2 The booking journey, end to end

THE SPINE OF ANY DEMO

1. **Project first.** A student registers a project; a lecturer approves it. Nothing can be booked against an unapproved project — the API refuses it outright.
2. **Workshop gate.** Booking only opens after workshop attendance is confirmed for that student (the booking page checks eligibility first).
3. **Book.** The student picks a kit bag or extras (they land in the cart), chooses shoot dates on the calendar, and pins the shooting location on the map (stored on the project).
4. **Submit.** POST /api/bookings — inside one database transaction the API re-checks every model for overlapping bookings and allocates real item rows (FOR UPDATE SKIP LOCKED), so two people can never grab the same unit at once.
5. **Review.** Staff see the pending booking in Reservation review; Approve is the gate that makes it visible to the hand-over desk.
6. **Check-out.** Staff inspect every item and capture four angle photos; the student signs with their own 4-digit studio PIN; a receipt is issued.
7. **Gate.** Security scans the student's pass; the booking record itself is the paperwork.
8. **Check-in.** Each item comes back with a condition (good / fair / damaged) or is flagged missing; one damaged item automatically creates a damage report, a maintenance log, a pending approval and an insurance claim.
9. **Aftermath.** Repairs list, approvals queue (staff propose → admin confirms), insurance hub (admin decides claims), notifications (overdue / short stock / late returns) and the statistics dashboard.

> **Five-minute classroom demo** (adapted from the project's own demo script): sign in as the student (*student@studio.edu*) → Book equipment: pick a kit, a shoot date, pin "Braamfontein" on the map → as staff (*staff@studio.edu*): Reservation review → Approve → Equipment check-out: angle photos + student PIN → check

one item in as **Damaged** → show it on Repairs & damage → then the request in Equipment approvals and the claim in Insurance hub → finish on **Shooting locations**, filtering the date to show was / is / will be shooting.

3 Web app — Team22-Main

REACT · VITE · TAILWIND

What it is: the React front-end that students, staff, lecturers, admins and security use in a browser. Routes live in App.tsx; each role's landing page and sidebar are derived from the same role map.

Q · What is the web app built with, and how does it talk to the backend?

React + TypeScript on Vite, styled with Tailwind and shadcn/ui. Shared state lives in five contexts (Role, Project, Student, Reservation, Equipment), and every HTTP call goes through one helper — **apiFetch()** in src/lib/api.ts — which attaches the bearer token. On the web the API base is empty, so all calls ride through nginx on the same origin.

Q · Walk me through a student booking a kit.

Four steps — Kit → Date → Terms → Done. Kit bags and extras go into the cart. Dates are picked on a hand-built calendar (no third-party booking widget): fully booked days are greyed out and a shoot is capped at three days. The shooting location is searched through the project's own /api/geo/search proxy, pinned on a map and saved on the project. Submitting sends POST /api/bookings — blocked earlier if the project isn't approved or the workshop hasn't been attended.

Q · How does the calendar know which days are already busy?

It asks the API for the overlapping bookings of everything in the cart (*/api/bookings/timeline?modelId=…&all=true*) and, for kit bags, the kit's booked windows (*/api/bookings/kit-bookings?kitid=…*). For extras, a day disables only when the requested quantity exceeds what's free; a kit blocks its whole booked window.

Q · What happens after the student submits?

The booking arrives as **pending** in the staff queue — Reservation review. Approve / Reject (PATCH /api/bookings/:id/approve or /reject) flips the status; only approved bookings appear at the hand-over desk.

Q · Describe check-out.

Three steps: Review → Inspect → Sign-off. Every item is ticked off and four angle photos are captured (front / back / left / right — re-shooting an angle replaces the old photo). The student accepts the terms and types their own 4-digit studio PIN, which proves they were present; the hand-over is confirmed and a receipt is issued.

Q · What does the Shooting locations page do?

It pins every project on a map and colours each pin for the selected date — "shooting on this date" (green), "will be shooting" (blue), "was shooting" (grey). It answers one operational question: where is our gear on day X. (War story: the date picker used to open *behind* the map because leaflet's panes paint up to z-index 800 — fixed by giving the map its own stacking context.)

Q · What does the Statistics page show?

KPI cards and charts (bookings, utilisation, damages) that follow a date filter — week / month / semester / all, plus a custom range. The numbers are computed from the same live data the other pages read, so there is no separate analytics database to drift out of sync. Each chart follows the "speak for themselves" pattern: a proof line naming the data behind it and a one-line verdict.

Q · Who sees what — and where is that enforced?

The sidebar, the landing page and ProtectedRoute all filter by role — lecturers land on project approvals, security lands on the scan screen, everyone else on the dashboard. The API re-checks every request, so hiding a button is never the only defence.

All 21 screens are documented one-by-one in the project's Docs/ folder — endpoints, tables and the reason each was built that way (start at Docs/README.md).

4 Mobile app — Team22-Mobile

CAPACITOR · ANDROID

What it is: the same React front-end, wrapped with Capacitor and built into an Android APK — so students can use the studio system from a phone on the local network.

Q · Is the mobile app a separate codebase?

No. It is the same front-end (the "Front end" folder mirrors the web app's src/), wrapped by Capacitor (appId *com.studio.booking*, webDir *dist*) and built into **StudioBooking-Hotspot.apk**. The android/ folder is the Gradle project; dist/ holds the built web bundle.

Q · How does the phone reach the API without nginx?

The build sets VITE_API_URL to the hotspot machine (e.g. *http://192.168.137.1:5000*) and apiFetch calls the API directly; on the web the same code leaves the base empty and rides through nginx instead. Photo URLs resolve through mediaUrl(), with mediaAlternateUrl() as a fallback that retries http://<host>:5000 when needed.

Q · How is it deployed for a class?

The PC runs the database + API containers (the mobile repo ships its own docker-compose.yml) and shares a hotspot; students connect to it and use the APK. Tip: test the Equipment page on the phone before a live demo — a blank Equipment screen was seen once, and the session investigating it (26 Aug) was interrupted before a fix was confirmed, so treat it as a check-before-demo item.

Q · Do photos and uploads work on the phone?

Yes — same endpoints and storage as the web: the phone uploads to the API host, and the returned /uploads/... paths resolve against that host.

5 API — Team22-API

NODE · EXPRESS 5 · POSTGRESQL

What it is: a Node/Express 5 application that owns every rule, all the SQL and all file uploads. server.js boots src/app.js, which mounts fourteen routers under /api and serves /uploads statically.

Q · What is the backend made of?

Express 5 with a single shared PostgreSQL pool (src/config/db.js — the only database connection), JWT auth, helmet + CORS + rate limiting, multer for uploads, pdfkit for PDFs/receipts, and a notification bot that starts with the server. The route files stay thin; all the SQL lives in the controllers.

Q · How is access controlled?

Every sensitive route runs **verifyToken** (a JWT "digital ticket" — 401 if it is missing, 403 if it is invalid) and then **requireRole([...])** against the database's role names (Studio Staff, Admin, Lecturer, ...). The maintenance/supervision area adds local guards: staff propose status changes, an admin records the decision. The UI mirrors these rules, but the API is the enforcement point.

Q · How is double-booking actually prevented?

Three cooperating layers: the calendar disables fully booked days before submission; kit windows expose the kit's existing bookings so an overlapping window is blacked out; and at submit time POST /api/bookings runs inside a transaction that re-checks overlaps and allocates physical item rows with **FOR UPDATE SKIP LOCKED**. If a unit was taken mid-request it is skipped; if a request cannot be fully satisfied the whole transaction rolls back — no half-bookings.

Q · How do kit bags differ from extras — and why don't kits say "not enough"?

A kit bag is a physical kit (kits table) built from a template (kit_types + a kit_type_items checklist; untracked lines like "AX11 has a fixed lens" never create inventory). Kits are booked as a unit **against the calendar**: a booking blocks the kit's dates, and another student can book the same bag for different days — only overlapping windows are blocked (returned / rejected / cancelled bookings are ignored). Extras are individual models where availability is counted per unit.

Q · What must be true before a booking is accepted?

The student exists, has workshop attendance confirmed, and owns or is on the team of a project whose status is **approved**. Each failure returns a specific message ("Workshop attendance has not been confirmed…", "This project has not been approved by your lecturer yet").

Q · What happens when a return is damaged or missing?

returnBooking normalises each item's condition; damaged items create a damage report, a maintenance log, a pending approval and an insurance claim in a single transaction, with the evidence photos linked onto the records (a shared helper resolves the right image; audit-damage-evidence.js fails if any record's photo URL is empty).

Q · Two best "under the hood" stories?

The geocoding proxy (/api/geo/search): the public provider throttles browser calls and the packaged phone app has no Referer, so the API does the lookup itself — user agent, 10-minute cache, ~1 request/second, Johannesburg bias. **The notification bot:** operational rules (overdue kits, short stock, late returns) become alerts in the in-app bell, checked every few minutes.

6 Database

POSTGRESQL 15

The database is created from numbered migrations — the Postgres container runs database/init.sql/*.sql in order the first time it boots on an empty volume.

Q · Where does the schema live, and how do I rebuild it?

Team22-API/database/init.sql/ — files **000 to 026**. 000 is the baseline schema; the rest are the migrations and seeds in the order the project grew (003–006 kits, 013/025 insurance, 018–021 kit bags vs extras, 022 the 4-digit PINs, 026 maintenance evidence photos). A new change = the next number; rebuilding from scratch is just a fresh volume + docker compose up.

Q · Which tables matter most?

Grouped: **Identity** — role, User, student / lecturer / studiostaff / admin / security. **Catalogue** — categories, equipmentmodels → equipmentitems (each serial-numbered unit), locations. **Kits** — kit_types, kit_type_items, kits. **Booking** — project, project_team, project_invites, bookings, bookingitems, gatepasses, bookingdocuments, cart_items, workshop_codes. **Workflow** — damagereports, maintenancelogs, pending_approval, insurance, insurance_claims, equipment_condition_images, equipmentnotifications.

Q · How does the database know what's available?

Two facts working together: each physical unit (equipmentitems) carries a **currentstatus** (available, on-set, in-repair, damaged, missing, …) and each booking line (bookingitems) carries expected / actual pickup and return timestamps. The calendars, the inventory page and the availability logic all read that pair.

7 Questions you'll get asked

QUICK ANSWERS

Q · "One student books a kit today; another wants the same kit for a different day — will the system say there isn't enough?"

No. Kit bags are booked against the calendar, not a stock counter: only overlapping dates are blocked, and the bag just has to be back before the next booking. Counters apply to extras.

Q · "Two students book the same camera for the same day — what happens?"

The second request finds no free unit for the overlapping dates inside its transaction. Nothing is half-saved (the transaction rolls back) and the student is told the equipment isn't available.

Q · "Where do I look to find our gear?"

Inventory: is it available. Tracked equipment: who has this specific serial unit, with photos and history. Booking history: what a project has out. Shooting locations: where productions are on any date.

Q · "Tell the life story of one dropped lens."

Checked in as **Damaged** → a damage report, maintenance log, approval request and insurance claim are created together → it appears in Repairs & damage → a staff member proposes "in-repair", an admin confirms → Insurance hub decides the claim and the register updates. Photos and timestamps exist at every step.

Q · "Who can do what?"

Student: projects, bookings, gate pass, own PIN. **Lecturer:** approve/reject projects, attendance. **Studio Staff:** approve bookings, check-out/in, inventory, repairs. **Admin:** insurance decisions, approval confirmations, accounts. **Security:** the scan screen only. (Enforced in the UI and again in the API.)

Q · "It's demo day and something dies — how do I recover?"

Restart the stack with *docker compose up -d* and check it with check-studio.ps1; the web opens on port 80, the API answers on :5000 (/api/health). For the phone, re-test the hotspot address before students connect.

8 How this was built — AI usage notes

HONEST RECORD

Short, factual notes on where AI assistance shows up, per part of the system — documented so it can be discussed in the open.

- **Web:** three real sessions shaped the Statistics page — 16 Aug (a full redesign, built then reverted on request), 20 Aug (the UX rework: chart-selector buttons, the proof/verdict pattern, quieter colours), 22 Aug (flashing-green positive recommendations).
- **Mobile & API:** the 26 Aug session investigated the blank Equipment page on the phone and the kit-bag booking logic; it was interrupted by provider errors before fixes were confirmed.
- **Scope note:** those are the four sessions on record. The system grew well beyond them, and the durable record of how anything works is the code itself plus the project's Docs/ folder — 21 page-by-page documents, a master code map and a calendar walkthrough. When in doubt: open the doc, open the file.

Compiled 4 Oct 2026 from the live codebases (Team22-API · Team22-Main · Team22-Mobile) and the project's own Docs/ folder. Companion document: the AI Usage Log covering the 16–26 Aug 2026 sessions.